Requirements Verification Artifacts

Requirement IDs 1–34 | Proposed draft for team/GTA review

SCOPE STATUS The workbook leaves Sprint No. blank for all 34 rows. This document covers the full requirements backlog;

Purpose

Turn each source requirement into a reviewable verification package: expected behavior, acceptance criteria, example inputs and expected outputs, constraints, dependencies, a proposed definition of done, and a place to attach real evidence. The example tests below are proposals only; none were executed, and no implementation results are claimed.

How to review this artifacts

- Each requirement section preserves the source row wording and adds a proposed, testable interpretation.
- Items that need a policy choice (such as currency rates, variance convention, precision, or supported equation class) call it out in the validation/open-decisions section.
- After the team agrees sprint scope, enter the approved sprint number in the google sheet and copy only those approved requirements into the Sprint 1 submission package.

Google Sheet: Project 3: Requirements Open source workbook

Requirement register

ID	Requirement	Points	Priority	Sprint No.
01	Expression parser	1	1	1
02	Arbitrary-precision calculation engine	1	2	1
03	Persistent session history and Ans recall	1	3	1
04	Unit conversion	2	4	1
05	Clean, user-friendly UI layout	2	5	1
06	2D single-variable graphing	2	6	1
07	Calculation mode switcher	1	7	1
08	Function search and category filter	1	8	1
09	Financial calculations	2	9	2
10	Calculation-step and formula breakdown	2	10	2
11	Helpful input checker	1	11	2
12	Undo and redo for expression editing	1	12	2
13	Number theory tools	3	13	2
14	Displayed precision controls	2	14	2
15	Tolerance calculator	2	15	2
16	Statistical calculations	3	16	2
17	Fraction calculator	3	17	2

ID	Requirement	Points	Priority	Sprint No.
18	Percentage calculator	2	18	2
19	Graph customization	3	19	2
20	Basic algebra equation solver	5	20	2
21	Favorite calculations	2	21	2
22	Display customization	2	22	2
23	Export calculation history	3	23	2
24	Graph comparison	3	24	2
25	Financial scenario comparison	3	25	2
26	Programmer mode	2	26	2
27	Matrix operations	2	27	2
28	Calculus tools	1	28	2
29	Complex-number mode	1	29	2
30	Find graph intercepts	2	30	Unassigned
31	Constants library	2	31	Unassigned
32	Save graphs as PNG	1	32	Unassigned
33	Notes on saved calculations	1	33	Unassigned
34	Light and dark mode	1	34	Unassigned

Requirement 09 — Financial calculations

Proposed / hypothetical • Story points: 2 • Priority: 9 • Sprint: 2

Source description: A standard financial calculation module covering compound interest, loan payments, etc.

What it should do

Calculate at least compound-interest growth and periodic loan payments using explicitly labeled financial inputs and assumptions.

Acceptance criteria

- Compound-interest results follow the selected rate/compounding convention.
- Loan payment results use the selected payment timing, rate period, and number of payments.
- Inputs and outputs show units/periods, with currency rounded only for display according to policy.

Example test cases — proposed, not run

Test ID	Input / action	Expected output
R09-T1	Principal $1,000; nominal annual rate 10%; monthly compounding; 1 year	Future value $1,104.71 to cents, using A=P(1+0.10/12)^12.
R09-T2	Loan $10,000; 6% nominal APR; monthly payments; 36 months; end-of-month payments	$304.22 monthly to cents, using monthly rate 0.06/12 and the standard amortizing-payment formula.

UI / workflow notes: Use labeled fields for principal, rate, term, compounding/payment frequency, and result. Show the selected assumptions beside the result.

Validation, limitations, and open decisions: Reject negative term, nonpositive payment count, invalid rate formats, and missing units. The source does not define APR vs effective rate, payment timing, fees, taxes, or rounding; examples above propose nominal annual rates and monthly periods.

Dependencies and risks: Needs numeric engine (ID 2), finance formulas, and possibly scenario comparison (ID 25). Risk: unstated conventions can create materially different answers.

Proposed definition of done: Finance conventions and formulas are team-approved; both examples and invalid-input cases pass; displayed assumptions and actual outputs are captured.

Evidence to add after verification: [Attach real test output, screenshot, demo link, build/commit, date, and observed result. Leave unfilled until evidence exists.]

Requirement 10 — Calculation-step and formula breakdown

Proposed / hypothetical • Story points: 2 • Priority: 10 • Sprint: 2

Source description: A breakdown view that shows the calculation steps, including the formula definitions when needed.

What it should do

Explain how a supported calculation reaches its result by showing ordered operations and, for named formulas, the formula definition with substituted values.

Acceptance criteria

- Steps are ordered consistently with the evaluator and precedence rules.
- The final displayed step agrees with the calculator result.
- A formula-based feature can show its selected formula and input substitution.

Example test cases — proposed, not run

Test ID	Input / action	Expected output
R10-T1	Request a breakdown for 2 + 3 × 4	Steps show 3 × 4 = 12, then 2 + 12 = 14.
R10-T2	Request the compound-interest breakdown for the R09-T1 inputs	Show A=P(1+r/n)^(nt), substitute the agreed values, and end at $1,104.71 to cents.

UI / workflow notes: Place a clearly labeled expandable steps view near the result; keep formula symbols and units readable.

Validation, limitations, and open decisions: Define which features must provide steps and how to handle operations whose internal algorithm is not naturally a human-readable sequence.

Dependencies and risks: Must use parser/evaluator semantics (IDs 1–2) and feature formulas (for example ID 9). Risk: a separately generated explanation can disagree with the actual computation.

Proposed definition of done: Breakdown is generated from the calculation's real operation/formula record; step order and final result match; representative evidence is attached.

Evidence to add after verification: [Attach real test output, screenshot, demo link, build/commit, date, and observed result. Leave unfilled until evidence exists.]

Requirement 11 — Helpful input checker

Proposed / hypothetical • Story points: 1 • Priority: 11 • Sprint: 2

Source description: An input checker with helpful explanations for issues like missing parentheses or division by zero.

What it should do

Detect common malformed or invalid expressions and explain the problem in terms the user can act on.

Acceptance criteria

- A missing parenthesis is identified with a corrective message.
- Division by zero is reported as an error, not a valid numeric result.
- An invalid expression does not crash the app or overwrite the prior valid result without explanation.

Example test cases — proposed, not run

Test ID	Input / action	Expected output
R11-T1	Evaluate 2 + (3	Show an unmatched/missing closing-parenthesis message; do not claim a result.
R11-T2	Evaluate 8 ÷ 0	Show a division-by-zero message; do not return infinity as an ordinary result.

UI / workflow notes: Place a concise message near the input; indicate the relevant token/location when available and preserve editable input for correction.

Validation, limitations, and open decisions: Agree error categories, location conventions, localization, and whether the last successful result remains visible but marked as previous.

Dependencies and risks: Depends on parser and numeric engine (IDs 1–2). Risk: duplicated error rules may disagree with actual evaluation.

Proposed definition of done: Each supported error has a reproducible trigger and actionable message; no crash or misleading result occurs; screenshots/logs of actual diagnostics are attached.

Evidence to add after verification: [Attach real test output, screenshot, demo link, build/commit, date, and observed result. Leave unfilled until evidence exists.]

Requirement 12 — Undo and redo for expression editing

Proposed / hypothetical • Story points: 1 • Priority: 12 • Sprint: 2

Source description: Undo and redo for expression editing.

What it should do

Let users reverse and reapply expression edits in the active editing context without undoing unrelated calculations or settings.

Acceptance criteria

- Undo restores the previous expression state.
- Redo reapplies the state that was undone.
- A new edit after undo follows the agreed redo-history clearing rule.

Example test cases — proposed, not run

Test ID	Input / action	Expected output
R12-T1	Type 2 + 3, change 3 to 4, Undo	Editor returns to 2 + 3.
R12-T2	Redo after R12-T1	Editor returns to 2 + 4.

UI / workflow notes: Provide discoverable undo/redo controls and keyboard shortcuts if agreed; disabled state should be clear when no action is available.

Validation, limitations, and open decisions: Define history depth and whether undo applies only to text edits, clipboard changes, or other editor actions.

Dependencies and risks: Uses the expression editor (ID 5) and mode behavior (ID 7). Risk: shortcut handling can conflict with app/browser/system conventions.

Proposed definition of done: Editing history follows the approved scope; undo/redo and branch-after-undo cases pass; the restored expression is visible and evidence is attached.

Evidence to add after verification: [Attach real test output, screenshot, demo link, build/commit, date, and observed result. Leave unfilled until evidence exists.]

Requirement 13 — Number theory tools

Proposed / hypothetical • Story points: 3 • Priority: 13 • Sprint: 2

Source description: Number theory tools: prime check, prime factorization, greatest common divisor, and least common multiple.

What it should do

Provide prime testing, integer factorization, greatest common divisor, and least common multiple for supported integer inputs.

Acceptance criteria

- Prime check distinguishes primes and composites for the supported integer domain.
- Factorization multiplies back to the original positive integer.
- GCD and LCM return mathematically consistent integer results.

Example test cases — proposed, not run

Test ID	Input / action	Expected output
R13-T1	Prime check 17 and 21	17 is prime; 21 is composite.
R13-T2	Factorize 60	$2^2 \times 3 \times 5$.
R13-T3	GCD(12,18), LCM(12,18)	6 and 36, respectively.

UI / workflow notes: Expose each operation with clearly labeled integer inputs and result format; show factors as a product with exponents or an agreed equivalent.

Validation, limitations, and open decisions: Define behavior for 0, 1, negative values, non-integers, and very large integers. Factorization runtime can grow quickly for large inputs.

Dependencies and risks: Uses integer arithmetic (ID 2) and function catalog/search (ID 8). Risk: edge-case definitions, especially factorization of zero/negative values, are absent.

Proposed definition of done: Input domain and edge cases are agreed; prime, factorization, GCD, and LCM cases pass, including selected boundaries; actual outputs are recorded.

Evidence to add after verification: [Attach real test output, screenshot, demo link, build/commit, date, and observed result. Leave unfilled until evidence exists.]

Requirement 14 — Displayed precision controls

Proposed / hypothetical • Story points: 2 • Priority: 14 • Sprint: 2

Source description: Precision controls for displayed decimal places or significant figures while preserving the full stored value

What it should do

Let users choose decimal-place or significant-figure formatting while retaining the unrounded calculation value for subsequent operations.

Acceptance criteria

- The chosen display mode and precision are applied consistently to results.
- Changing display precision does not mutate the stored result.
- Subsequent calculations use the full stored value, not the rounded text.

Example test cases — proposed, not run

Test ID	Input / action	Expected output
R14-T1	Calculate 1 ÷ 3; show 2 decimal places, then 4	Display changes from 0.33 to 0.3333.
R14-T2	After calculating 1 ÷ 3, calculate Ans × 3	Result is 1 within the documented engine precision, regardless of the earlier display formatting.

UI / workflow notes: Offer a labeled precision control and make the active decimal-place/significant-figure mode visible.

Validation, limitations, and open decisions: Agree rounding mode, significant-figure handling for zero, maximum precision, and how precision is represented in exports/history.

Dependencies and risks: Depends on numeric engine (ID 2), history/Ans (ID 3), and formatting shared by financial/statistical results. Risk: rounded display text may be reused as data.

Proposed definition of done: Formatting rules are approved; visual formatting and full-value reuse tests pass; evidence shows both displayed forms and the unchanged computed value.

Evidence to add after verification: [Attach real test output, screenshot, demo link, build/commit, date, and observed result. Leave unfilled until evidence exists.]

Requirement 15 — Tolerance calculator

Proposed / hypothetical • Story points: 2 • Priority: 15 • Sprint: 2

Source description: A tolerance calculator that computes minimum and maximum results from measurements with specified tolerances.

What it should do

For a measurement with a nominal value and tolerance, report the allowable minimum and maximum under the agreed tolerance-combination rules.

Acceptance criteria

- For a single symmetric tolerance, minimum = nominal − tolerance and maximum = nominal + tolerance.
- Units are retained or converted consistently.
- Invalid negative tolerance is rejected or explained according to the approved rule.

Example test cases — proposed, not run

Test ID	Input / action	Expected output
R15-T1	Length 10.0 mm ± 0.2 mm	Minimum 9.8 mm; maximum 10.2 mm.
R15-T2	Nominal 5, tolerance 0	Minimum 5; maximum 5.

UI / workflow notes: Provide nominal value, tolerance, and unit inputs; display the interval as [minimum, maximum] and show units.

Validation, limitations, and open decisions: Define supported measurement operations and how tolerances combine for sums, products, correlated variables, asymmetric tolerances, and rounding.

Dependencies and risks: Uses numeric arithmetic and possibly unit conversion (IDs 2 and 4). Risk: independent worst-case intervals may not reflect correlated measurements.

Proposed definition of done: The supported tolerance model is documented; boundary and unit cases pass; interval values and assumptions are visible in captured evidence.

Evidence to add after verification: [Attach real test output, screenshot, demo link, build/commit, date, and observed result. Leave unfilled until evidence exists.]

Requirement 16 — Statistical calculations

Proposed / hypothetical • Story points: 3 • Priority: 16 • Sprint: 2

Source description: A statistical calculation module that allows users to calculate mean, median, mode, standard deviation, and variance for a given dataset.

What it should do

Accept a dataset and compute the five listed descriptive statistics using a documented population/sample convention.

Acceptance criteria

- Mean, median, mode, standard deviation, and variance are calculated from the same validated dataset.
- Empty and malformed datasets produce an explanatory result instead of a numeric claim.
- The variance convention and behavior when multiple/no modes exist are visible or documented.

Example test cases — proposed, not run

Test ID	Input / action	Expected output
R16-T1	Dataset [1, 2, 2, 3]; proposed population convention	Mean 2; median 2; mode 2; population variance 0.5; population standard deviation ≈0.7071.
R16-T2	Dataset []	Explain that statistics cannot be computed for an empty dataset.
R16-T3	Dataset [1, 2, 3, 4]; proposed population convention	Mean 2.5; median 2.5; population variance 1.25; standard deviation ≈1.1180; mode behavior must follow the team's selected rule.

UI / workflow notes: Offer a dataset entry workflow (for example comma-separated values) and display each statistic with a clear label.

Validation, limitations, and open decisions: The source does not choose sample vs population variance, mode behavior for ties/no mode, accepted separators, or missing-value handling. Tests above propose population variance.

Dependencies and risks: Needs numeric engine (ID 2), dataset parser, and precision formatting (ID 14). Risk: an undocumented convention produces a numerically plausible but incompatible answer.

Proposed definition of done: Dataset syntax and all statistical conventions are approved; known-value, tie/no-mode, empty, and invalid-input cases pass; actual outputs are recorded.

Evidence to add after verification: [Attach real test output, screenshot, demo link, build/commit, date, and observed result. Leave unfilled until evidence exists.]

Requirement 17 — Fraction calculator

Proposed / hypothetical • Story points: 3 • Priority: 17 • Sprint: 2

Source description: A fraction calculator that allows users to perform arithmetic operations with fractions and convert between fractions and decimals.

What it should do

Perform the agreed arithmetic operations on fractions and convert between fractional and decimal representations.

Acceptance criteria

- Fraction arithmetic returns an equivalent correctly reduced fraction under the approved formatting rule.
- Fraction/decimal conversion preserves value within the selected decimal precision.
- Zero denominators are rejected with a clear message.

Example test cases — proposed, not run

Test ID	Input / action	Expected output
R17-T1	1/2 + 1/3	5/6.
R17-T2	0.125 → fraction	1/8.
R17-T3	1/0	Clear zero-denominator error; no numeric result.

UI / workflow notes: Allow fraction entry and show both exact fraction and optional decimal value where useful.

Validation, limitations, and open decisions: Define mixed-number and negative syntax, reduction rules, recurring decimal handling, and precision for decimal-to-fraction conversion.

Dependencies and risks: Uses parser and numeric/rational arithmetic (IDs 1–2), and may share display precision rules (ID 14). Risk: decimal-to-fraction conversion may have many valid approximations.

Proposed definition of done: Fraction syntax and reduction policy are approved; arithmetic, conversion, and denominator errors pass; exact and displayed forms are captured.

Evidence to add after verification: [Attach real test output, screenshot, demo link, build/commit, date, and observed result. Leave unfilled until evidence exists.]

Requirement 18 — Percentage calculator

Proposed / hypothetical • Story points: 2 • Priority: 18 • Sprint: 2

Source description: A percentage calculator that allows users to calculate percentage increases, decreases, and differences between values.

What it should do

Calculate percentage increase, decrease, and difference using explicitly labeled starting and comparison values.

Acceptance criteria

- Increase and decrease use the starting value as the denominator.
- The result states the direction and percent unit.
- Percentage-difference denominator is defined by an approved convention.

Example test cases — proposed, not run

Test ID	Input / action	Expected output
R18-T1	Increase from 80 to 100	25% increase.
R18-T2	Decrease from 100 to 80	20% decrease.
R18-T3	Difference between 80 and 100; proposed symmetric convention	22.22% = |100−80| / ((100+80)/2) × 100, rounded to 2 decimals.

UI / workflow notes: Offer separate labeled workflows for increase, decrease, and difference so the baseline is unambiguous.

Validation, limitations, and open decisions: Define percentage-difference convention, behavior for zero/negative baselines, and rounding. R18-T3 is a proposal that requires team agreement.

Dependencies and risks: Uses numeric engine and display formatting (IDs 2 and 14). Risk: users may mistake symmetric percentage difference for percent increase from a baseline.

Proposed definition of done: Each workflow labels its denominator/convention; examples and zero-baseline cases follow the documented rule; evidence shows direction and units.

Evidence to add after verification: [Attach real test output, screenshot, demo link, build/commit, date, and observed result. Leave unfilled until evidence exists.]

Requirement 19 — Graph customization

Proposed / hypothetical • Story points: 3 • Priority: 19 • Sprint: 2

Source description: A graph customization feature that allows users to adjust axis ranges, gridlines, and graph labels.

What it should do

Let users set visible axis ranges, show or hide gridlines, and edit graph labels without changing the underlying plotted equations.

Acceptance criteria

- Valid manual axis bounds are reflected in the viewport.
- Gridline visibility follows the selected setting.
- Edited labels appear on the graph and remain associated with the intended axis/graph.

Example test cases — proposed, not run

Test ID	Input / action	Expected output
R19-T1	Plot y=x; set x and y ranges to −2…2	Both axes show the selected bounds and the line is clipped/displayed within that viewport.
R19-T2	Turn grid off, rename graph to “y=x demo”	Gridlines disappear and the graph label reads “y=x demo”.

UI / workflow notes: Put range, gridline, and label controls near the graph; show when manual ranges override automatic scaling.

Validation, limitations, and open decisions: Reject nonnumeric bounds and min ≥ max; define range units, tick spacing, and whether custom settings persist per graph.

Dependencies and risks: Builds on graphing (ID 6) and supports graph comparison/export (IDs 24 and 32). Risk: independent settings across multiple graphs can be unclear.

Proposed definition of done: Axis, grid, and label rules are approved; valid/invalid controls update only the intended graph; screenshots show each configured state.

Evidence to add after verification: [Attach real test output, screenshot, demo link, build/commit, date, and observed result. Leave unfilled until evidence exists.]

Requirement 20 — Basic algebra equation solver

Proposed / hypothetical • Story points: 5 • Priority: 20 • Sprint: 2

Source description: An equation solver that allows users to enter basic algebraic equations and solve for an unknown variable.

What it should do

Parse a team-approved class of basic algebra equations and return the solution for the requested unknown, including clear no-solution or multiple-solution outcomes where applicable.

Acceptance criteria

- A solvable supported equation returns a value satisfying the original equation.
- The solution is associated with the correct variable.
- Inconsistent and identity equations are distinguished if included in the supported scope.

Example test cases — proposed, not run

Test ID	Input / action	Expected output
R20-T1	Solve 2x + 3 = 11 for x	x = 4; substitution gives 11 = 11.
R20-T2	Solve x = x + 1	No solution.
R20-T3	Solve x = x	Every allowed x is a solution (identity), if identities are in the approved scope.

UI / workflow notes: Show the equation, selected unknown variable, and solution status; preserve the original equation for verification.

Validation, limitations, and open decisions: Define supported degree, variable syntax, implicit multiplication, domains, and treatment of nonlinear/multiple/complex solutions. “Basic” is not a precise equation grammar.

Dependencies and risks: Needs expression parser (ID 1), symbolic or equation-solving logic, and numeric engine (ID 2). Risk: unsupported equations may be mistaken for unsolvable equations.

Proposed definition of done: Equation class and solution semantics are agreed; solvable, inconsistent, identity, and unsupported cases are correctly distinguished; evidence includes substitution checks.

Evidence to add after verification: [Attach real test output, screenshot, demo link, build/commit, date, and observed result. Leave unfilled until evidence exists.]

Requirement 21 — Favorite calculations

Proposed / hypothetical • Story points: 2 • Priority: 21 • Sprint: 2

Source description: A favorite systems that lets users save frequently used calculations for quick access.

What it should do

Let users save a calculation to a favorites collection and retrieve it later without retyping its expression.

Acceptance criteria

- A user can add and identify a favorite calculation.
- Selecting a favorite restores the saved expression or calculation using the approved recall behavior.
- Saved favorites remain available across the agreed persistence boundary.

Example test cases — proposed, not run

Test ID	Input / action	Expected output
R21-T1	Save 2 + 3 as “Simple sum”, then reopen favorites	“Simple sum” appears with its expression; selecting it restores 2 + 3 and its result is 5 if results are stored.
R21-T2	Save the same expression twice	Duplicate handling follows the selected policy; no silent overwrite occurs.

UI / workflow notes: Provide a favorite action and a list with descriptive names; make recall distinct from immediate execution if that is the chosen behavior.

Validation, limitations, and open decisions: Define naming, duplicate handling, edit/delete, storage scope, and whether a favorite stores an expression, result, mode, or all of them.

Dependencies and risks: May reuse history records (ID 3), persistence, and mode context (ID 7). Risk: recalled calculations may depend on a changed rate or setting.

Proposed definition of done: Saved fields and recall semantics are agreed; save/recall/duplicate cases pass; evidence shows the item in the list and its restored calculation.

Evidence to add after verification: [Attach real test output, screenshot, demo link, build/commit, date, and observed result. Leave unfilled until evidence exists.]

Requirement 22 — Display customization

Proposed / hypothetical • Story points: 2 • Priority: 22 • Sprint: 2

Source description: Customization that lets users change colors, labels, and display settings.

What it should do

Let users adjust approved visual settings and labels, then apply those choices consistently to the relevant calculator views.

Acceptance criteria

- A supported color or label setting changes the intended view.
- A display setting changes its documented presentation behavior.
- Settings do not alter stored calculation values.

Example test cases — proposed, not run

Test ID	Input / action	Expected output
R22-T1	Change a graph/accent color and label	The selected color and label appear in the target view.
R22-T2	Change a display setting, then evaluate 2 + 3	Presentation follows the setting and the numeric result remains 5.

UI / workflow notes: Provide a settings area with named controls, current values, and a way to restore defaults if approved.

Validation, limitations, and open decisions: Define which colors/labels/display settings are customizable, persistence, allowed ranges, and accessibility contrast requirements.

Dependencies and risks: Uses shared theme/display tokens and UI (ID 5); may overlap with graph labels (ID 19) and light/dark mode (ID 34). Risk: settings may conflict or apply inconsistently.

Proposed definition of done: Customization scope and persistence are agreed; changed/default settings render correctly without affecting calculations; screenshots and setting values are captured.

Evidence to add after verification: [Attach real test output, screenshot, demo link, build/commit, date, and observed result. Leave unfilled until evidence exists.]

Requirement 23 — Export calculation history

Proposed / hypothetical • Story points: 3 • Priority: 23 • Sprint: 2

Source description: An export tool that lets users download their calculation history as a text or CSV.

What it should do

Export selected or complete calculation history as a text file or CSV that preserves the agreed record fields and can be read outside the app.

Acceptance criteria

- Text and CSV exports contain the expected history entries.
- CSV fields remain correctly delimited/escaped for commas, quotes, and line breaks.
- Exported values correspond to the source history and are not changed by display formatting unless specified.

Example test cases — proposed, not run

Test ID	Input / action	Expected output
R23-T1	History: 2+3 → 5 and 7×6 → 42; export CSV	Two data rows preserve expressions and results; proposed columns are timestamp, expression, result (confirm with team).
R23-T2	Export a record whose expression or note contains a comma	The CSV parses as one record with the comma-containing field intact.

UI / workflow notes: Offer text/CSV choice, a clear export action, and completion/failure feedback; avoid implying a download succeeded until confirmed.

Validation, limitations, and open decisions: Define included fields, timestamp format/time zone, character encoding, file naming, selection scope, and privacy expectations.

Dependencies and risks: Depends on history records (ID 3), possibly notes (ID 33), and file export support. Risk: ambiguous or incorrectly escaped data is hard to verify downstream.

Proposed definition of done: Export formats and fields are approved; representative and special-character files parse correctly; actual exported file evidence is attached.

Evidence to add after verification: [Attach real test output, screenshot, demo link, build/commit, date, and observed result. Leave unfilled until evidence exists.]

Requirement 24 — Graph comparison

Proposed / hypothetical • Story points: 3 • Priority: 24 • Sprint: 2

Source description: An graph comparison view that lets multiple equations be displayed either seperately or overlapping simulatenously.

What it should do

Display multiple selected equations in separate graph panes or together in one overlaid coordinate view so users can compare their shapes.

Acceptance criteria

- Each selected equation is identifiable and plotted in the selected comparison mode.
- Separate mode gives each graph a readable pane; overlay mode places graphs on shared axes.
- Changing comparison mode does not lose the selected equations.

Example test cases — proposed, not run

Test ID	Input / action	Expected output
R24-T1	Compare y=x and y=−x in separate mode	Two labeled panes show the rising and falling lines.
R24-T2	Switch the same two functions to overlay mode	Both lines appear on shared axes with distinguishable labels/styles.

UI / workflow notes: Show equation labels/legend and a clear separate/overlay control; maintain distinguishable line styling.

Validation, limitations, and open decisions: Define maximum graph count, shared vs independent axis scaling, color accessibility, and behavior for incompatible domains/ranges.

Dependencies and risks: Builds on graphing (ID 6), customization (ID 19), and UI settings (ID 22). Risk: overlapping plots can obscure each other or confuse axis scales.

Proposed definition of done: Comparison modes and scale rules are agreed; both functions remain identifiable in each mode; evidence includes separate and overlay screenshots.

Evidence to add after verification: [Attach real test output, screenshot, demo link, build/commit, date, and observed result. Leave unfilled until evidence exists.]

Requirement 25 — Financial scenario comparison

Proposed / hypothetical • Story points: 3 • Priority: 25 • Sprint: 2

Source description: A scenario feature that allows users to save multiple financial calculations and compare results side by side.

What it should do

Save named financial input sets and results, then compare their outputs side by side using the same clearly labeled fields.

Acceptance criteria

- A scenario preserves its inputs, assumptions, and calculated result.
- At least two scenarios can be viewed together with matching fields aligned.
- Changing one scenario does not silently change another.

Example test cases — proposed, not run

Test ID	Input / action	Expected output
R25-T1	Save $1,000 at 5% annually for 2 years and $1,000 at 7% for 2 years, annual compounding	Scenario A = $1,102.50; B = $1,144.90; side-by-side labels show rates and terms.
R25-T2	Edit the rate in Scenario B after saving both	Scenario A retains its original 5% inputs and $1,102.50 result.

UI / workflow notes: Provide named scenario cards or columns with aligned input, assumption, and result labels; make save/compare actions explicit.

Validation, limitations, and open decisions: Define which finance calculations can be saved, maximum scenarios, duplicate names, edit/delete, and whether saved results recalculate when formulas/rates change.

Dependencies and risks: Depends on financial calculations (ID 9), persistent storage, and possibly favorites (ID 21). Risk: comparing results based on different assumptions without surfacing those assumptions.

Proposed definition of done: Scenario fields and update rules are agreed; save, compare, and isolation cases pass; captured evidence shows inputs and outputs together.

Evidence to add after verification: [Attach real test output, screenshot, demo link, build/commit, date, and observed result. Leave unfilled until evidence exists.]

Requirement 26 — Programmer mode

Proposed / hypothetical • Story points: 2 • Priority: 26 • Sprint: 2

Source description: A programmer mode that allows users to convert numbers between decimal, binary, hex, and use bitwise logic.

What it should do

Convert integer values among decimal, binary, and hexadecimal representations and perform the approved bitwise operations.

Acceptance criteria

- Conversions preserve the same integer value across supported bases.
- Each supported bitwise operation returns the correct result under the selected width/signedness policy.
- Invalid digits for a base are rejected clearly.

Example test cases — proposed, not run

Test ID	Input / action	Expected output
R26-T1	Convert decimal 10 to binary and hexadecimal	Binary 1010; hexadecimal A.
R26-T2	Binary 1010 AND binary 1100	Binary 1000 (decimal 8) under ordinary bitwise AND.

UI / workflow notes: Make the active base visible beside input and output; label bitwise operation and operand base.

Validation, limitations, and open decisions: Define integer width, signedness, negative-number representation, leading zeros, overflow, and supported bitwise operators.

Dependencies and risks: Uses integer parsing/formatting and mode switcher (IDs 2 and 7). Risk: outputs differ substantially between signed/unsigned or fixed/unbounded widths.

Proposed definition of done: Base and bit-width rules are approved; conversions, bitwise operations, invalid digits, and boundary values pass; actual representations are captured.

Evidence to add after verification: [Attach real test output, screenshot, demo link, build/commit, date, and observed result. Leave unfilled until evidence exists.]

Requirement 27 — Matrix operations

Proposed / hypothetical • Story points: 2 • Priority: 27 • Sprint: 2

Source description: A matrix tool that allows users to add, multiply, and find the inverse.

What it should do

Accept valid matrices and perform addition, multiplication, and inverse operations where the required dimensions and invertibility conditions hold.

Acceptance criteria

- Addition requires matching dimensions and adds corresponding elements.
- Multiplication requires compatible dimensions and uses row-by-column products.
- Inverse is returned only for a square nonsingular matrix.

Example test cases — proposed, not run

Test ID	Input / action	Expected output
R27-T1	A=[[1,2],[3,4]], B=[[5,6],[7,8]]; A+B	[[6,8],[10,12]].
R27-T2	A×B for the same matrices	[[19,22],[43,50]].
R27-T3	Inverse of A=[[1,2],[3,4]]	[[-2,1],[3/2,−1/2]]; $A \times A^{-1}$ = identity.
R27-T4	Inverse of [[1,2],[2,4]]	Singular-matrix error; no inverse.

UI / workflow notes: Provide a clear matrix-entry grid, show dimensions, and format output rows/columns consistently.

Validation, limitations, and open decisions: Reject ragged matrices and incompatible dimensions. Define maximum size, numeric precision, and how near-singular matrices are handled.

Dependencies and risks: Uses numeric engine (ID 2), matrix input UI, and display precision (ID 14). Risk: rounding near singularity may affect inverse decisions.

Proposed definition of done: Dimension and singularity rules are documented; operation, identity-check, and invalid-dimension cases pass; actual matrices and precision are recorded.

Evidence to add after verification: [Attach real test output, screenshot, demo link, build/commit, date, and observed result. Leave unfilled until evidence exists.]

Requirement 28 — Calculus tools

Proposed / hypothetical • Story points: 1 • Priority: 28 • Sprint: 2

Source description: A calculus tool that allows users to find the derivatives and definite integrals.

What it should do

Compute derivatives and definite integrals for the team-approved class of functions and display the result with its variable and bounds.

Acceptance criteria

- A supported derivative result matches the approved symbolic or numeric convention.
- A definite integral uses the entered bounds and reports units/precision as applicable.
- Unsupported functions or invalid bounds produce an explanatory result.

Example test cases — proposed, not run

Test ID	Input / action	Expected output
R28-T1	Differentiate $f(x)=x^2$	2x under the proposed exact symbolic behavior.
R28-T2	Evaluate $\int_0^1 x^2\,dx$	1/3 under the proposed exact symbolic behavior; a numeric implementation may use an agreed tolerance.

UI / workflow notes: Separate derivative input from definite-integral input; show variable and integration bounds clearly.

Validation, limitations, and open decisions: Define supported functions, symbolic vs numeric behavior, numerical tolerance, singularity handling, and integration domain. The sample outputs assume exact elementary symbolic cases.

Dependencies and risks: Needs parser/function library (ID 1), numeric engine (ID 2), and possibly a symbolic algebra or numerical method. Risk: numerical approximations need a visible tolerance.

Proposed definition of done: Supported function class and accuracy policy are approved; known derivative/integral and invalid-domain cases pass; evidence includes method and actual result.

Evidence to add after verification: [Attach real test output, screenshot, demo link, build/commit, date, and observed result. Leave unfilled until evidence exists.]

Requirement 29 — Complex-number mode

Proposed / hypothetical • Story points: 1 • Priority: 29 • Sprint: 2

Source description: Complex number mode that allows users to do standard math using real and imaginary numbers.

What it should do

Evaluate the team-approved standard arithmetic operations for values with real and imaginary components and display results in a consistent complex form.

Acceptance criteria

- Addition and multiplication combine real and imaginary components correctly.
- The display distinguishes the imaginary unit from a variable or ordinary text.
- Division by zero complex value is reported safely if division is included.

Example test cases — proposed, not run

Test ID	Input / action	Expected output
R29-T1	(1+2i) + (3−i)	4 + i.
R29-T2	(1+2i) × (3−i)	5 + 5i.

UI / workflow notes: Make complex input syntax and result format visible; preserve signs and omit zero components only according to a documented format.

Validation, limitations, and open decisions: Define supported operations, syntax (i or j), parsing precedence, display precision, and whether roots/logarithms are in scope.

Dependencies and risks: Needs parser and numeric engine support (IDs 1–2), with separate complex arithmetic semantics. Risk: syntax conflicts with variable names or scientific notation.

Proposed definition of done: Complex syntax and operation set are approved; arithmetic and error cases pass; examples show both components and actual outputs.

Evidence to add after verification: [Attach real test output, screenshot, demo link, build/commit, date, and observed result. Leave unfilled until evidence exists.]